VERILOG TRAFFIC LIGHTS REPORT

Digital Design (25 points)	Verilog Implementation (25 points)	Verilog Testbench (25 points)	Academic Report and Presentation (25 points)	Total (100 points)

	Student Name and Surname	Student ID	Signature
1			
2			
3			
4			
5			
6			
7			
8			

Github link: https://github.com/benanozkor/traffic-light-verilog

1. INTRODUCTION

In this project, we designed and implemented a traffic light controller using Verilog hardware description language. The goal of the project was to understand basic digital design concepts such as finite state machines, counters, and simulation using a testbench.

The traffic light system controls four directions: North, East, South, and West. Each direction gets a green signal for a certain time, followed by a short all-red state before switching to the next direction. The system also includes a night mode feature where all lights blink to simulate low-traffic conditions.

2. SYSTEM DESIGN

The system is based on a Finite State Machine. Each state represents a specific condition of the traffic lights.

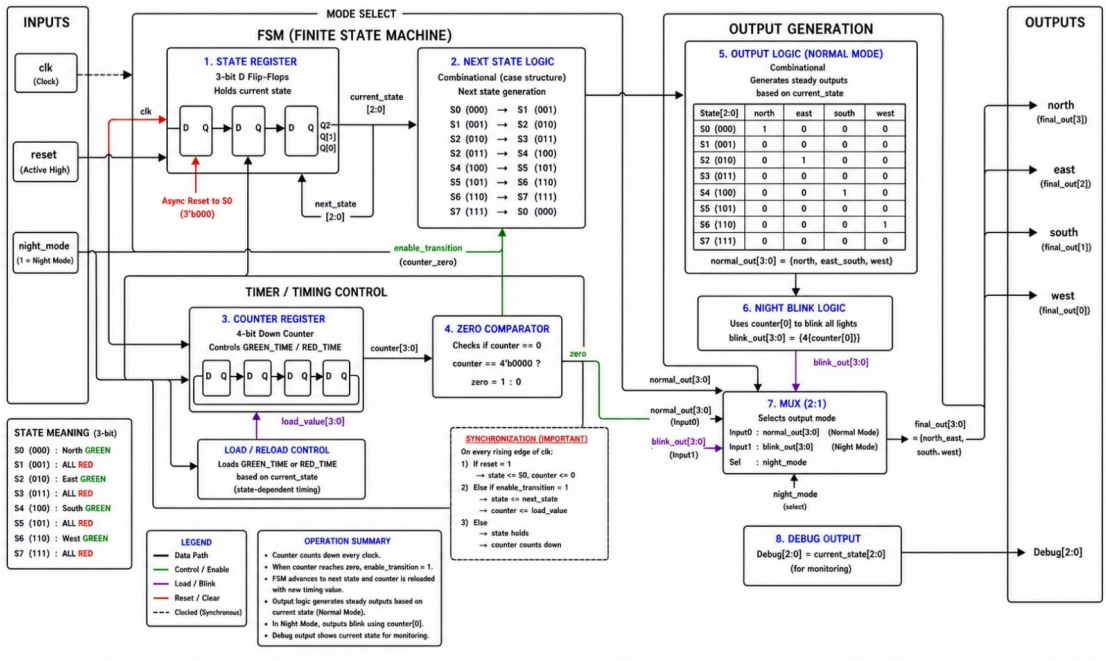
States:
S0 – North Green
S1 – All Red
S2 – East Green
S3 – All Red
S4 – South Green
S5 – All Red
S6 – West Green
S7 – All Red

The FSM transitions from one state to the next based on a clock signal and a counter. The counter determines how long each state lasts.

Timing:
- Green state duration: 3 clock cycles
- All-red state duration: 1 clock cycle

This ensures safe switching between directions.

3. DIGITAL DESIGN



As shown in the digital design schematic, the system consists of three main hardware blocks: FSM, Timer Control, and Output Logic. The FSM includes a 3-bit state register and combinational logic that determines the sequence of traffic lights. State transitions occur only when the timer reaches zero. The Timer Control block uses a 4-bit down counter to manage the duration of each state, reloading predefined values for green and red phases. The Output Logic generates traffic signals based on the current state. In addition, a 2:1 multiplexer is used to implement the night mode feature, which overrides normal operation and makes all lights blink simultaneously using the counter[0] signal. All components operate synchronously with the clock to ensure stable and correct system behavior.

4. VERILOG IMPLEMENTATION

The design was implemented in Verilog using three main components:

1. State Register:
   Stores the current state and updates on each clock edge.
2. Counter:
   Controls how long each state lasts by counting down clock cycles.
3. Output Logic:
   Determines which traffic light is ON based on the current state.

Signal Encoding:
- 3'b001 = Green
- 3'b100 = Red

5. TESTBENCH AND SIMULATION

A testbench was created to simulate the design and verify its behaviour.

The testbench performs the following:
- Generates a clock signal
- Applies reset at the beginning
- Runs the system in normal mode
- Activates night mode after some time
- Returns to normal mode

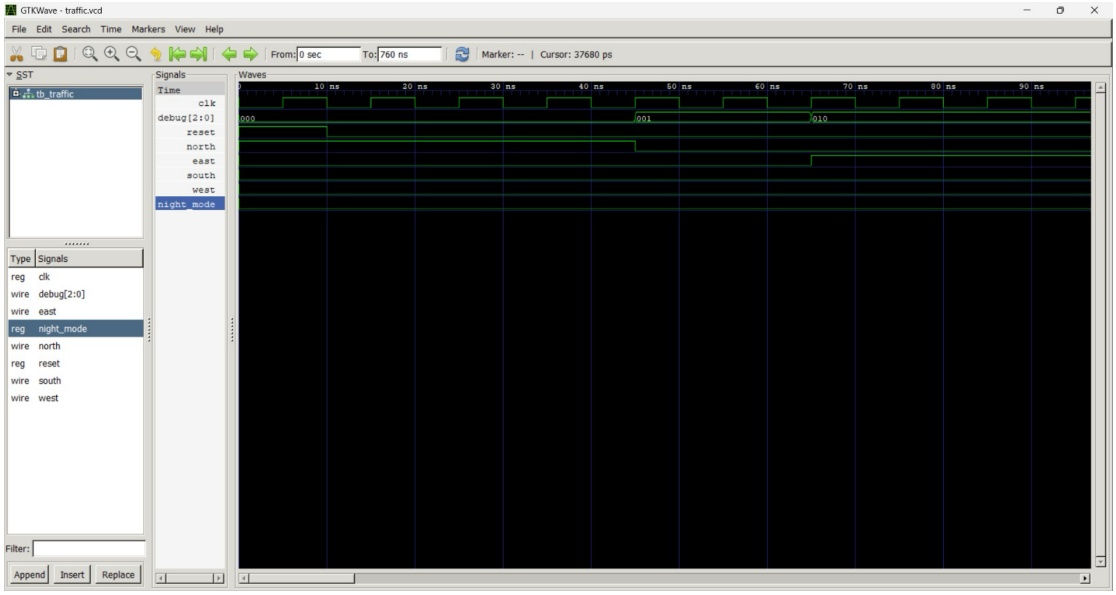
6. WAVEFORM ANALYSIS

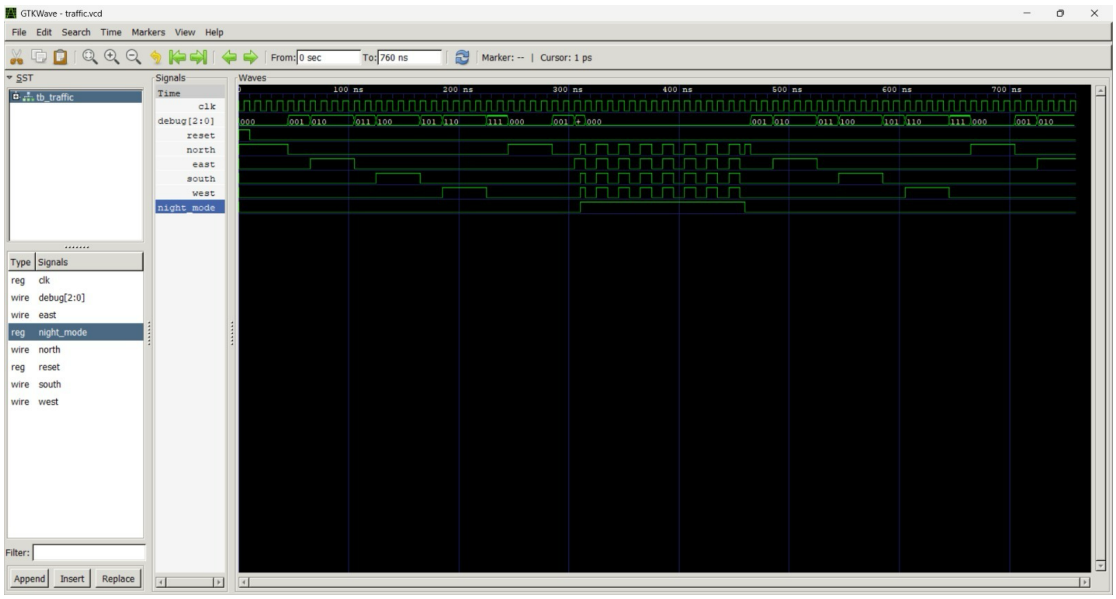
The waveform shows how signals change over time.
Important signals:

- clk: The clock signal driving the system
- debug: Shows the current FSM state
- north, east, south, west: Output signals for traffic lights

Results:
- The debug signal cycles through values 0 to 7, representing FSM states
- Each green state lasts longer than the red state
- Outputs change according to the current state
- During night mode, signals toggle repeatedly imitating blinking.

This confirms that the FSM and timing logic are working correctly.





7. NIGHT MODE

An additional feature called night mode was implemented.

When night_mode = 1:
- The system stops normal operation
- All traffic lights blink simultaneously

8. VERİLOG CODES

tb_traffic_light.v

```
`timescale 1ns/1ps

module tb_traffic;

reg clk;
reg reset;
reg night_mode;

wire north, east, south, west;
wire [2:0] debug;

// Instantiate
traffic_light_controller uut (
    .clk(clk),
    .reset(reset),
    .night_mode(night_mode),
    .north(north),
    .east(east),
    .south(south),
    .west(west),
    .debug(debug)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("traffic.vcd");
    $dumpvars(0, tb_traffic);
end

initial begin
    clk = 0;
    reset = 1;
    night_mode = 0;

    #10 reset = 0;
    #300;

    night_mode = 1;
    #150;

    night_mode = 0;
    #300;

    $finish;
end

endmodule
```

traffic_light_controller.v

```
module traffic_light_controller (
    input clk,
    input reset,
    input night_mode,
    output reg north,
    output reg east,
    output reg south,
    output reg west,
    output [2:0] debug
);
reg [2:0] state;
reg [2:0] next_state;
reg [3:0] counter;

parameter S0=0, S1=1, S2=2, S3=3,
          S4=4, S5=5, S6=6, S7=7;

parameter GREEN_TIME = 3;
parameter RED_TIME   = 1;

assign debug = state;


always @(posedge clk or posedge reset)
begin
    if (reset)
        state <= S0;
    else if (night_mode)
        state <= S0;
    else if (counter == 0)
        state <= next_state;
end

always @(posedge clk or posedge reset)
begin
    if (reset)
        counter <= GREEN_TIME;
    else if (counter == 0) begin
        case(state)
            S0, S2, S4, S6: counter <=
RED_TIME;
            default:        counter <=
GREEN_TIME;
        endcase
    end
    else
        counter <= counter - 1;
end

always @(*) begin
    case(state)
        S0: next_state = S1;
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = S4;
        S4: next_state = S5;
        S5: next_state = S6;
        S6: next_state = S7;
        S7: next_state = S0;
        default: next_state = S0;
    endcase
end

always @(*) begin
    // default = RED
    north = 0;
    east  = 0;
    south = 0;
    west  = 0;

    if (night_mode) begin
        // blink all lights
        north = counter[0];
        east  = counter[0];
        south = counter[0];
        west  = counter[0];
    end
    else begin
        case(state)
            S0: north = 1;
            S2: east  = 1;
            S4: south = 1;
            S6: west  = 1;
        endcase
    end
end

endmodule
```